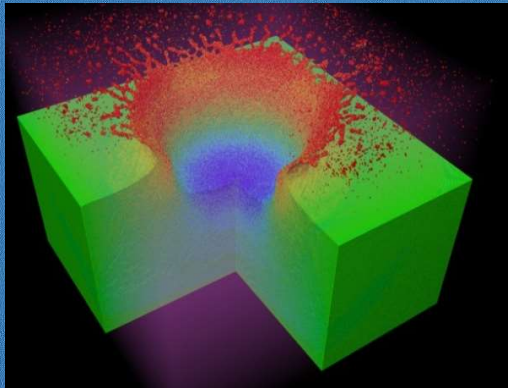


Sistemas Complejos

LCC
FING
UNCUYO



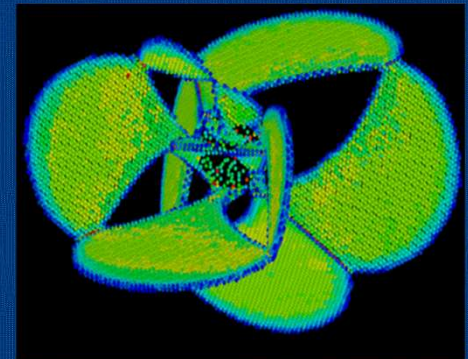
Eduardo M. Bringa
ebringa@yahoo.com

CONICET

FING, Universidad de
Cuyo, Argentina

<https://sites.google.com/site/simafweb>

Simulaciones de autómatas celulares



SiMAF

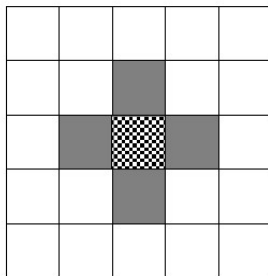
Outline

- CA: Definition, context and history.
- 1D CA and their classification
- 2D CA: Game of Life (GoL)
- Generalization and applications of CA
- Computational issues, demonstrated by two examples: GoL and settlement dynamics.
- Software for CA simulations: focus on *NetLogo* and additional examples.
- Bibliography: articles and web sites
- Computational Lab

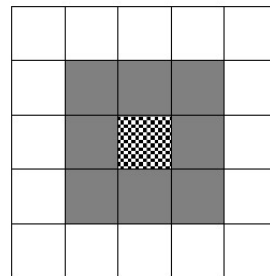
Cellular automaton (pl. cellular automata, CA)

http://en.wikipedia.org/wiki/Cellular_automata

- Discrete model: regular grid of *cells* (in N dimensions), each in one of a finite number of states, such as "On" and "Off".
- For each cell, define a *neighborhood*
- Discrete time. An initial state (time $t=0$) is selected by assigning a state for each cell.
- A new *generation* is created (advancing t by 1), according to some fixed “local” rule (generally, a mathematical function) that determines the new state of each cell in terms of the current state of the cell and the states of the cells in its neighborhood.



Von Neumann's
neighborhoods



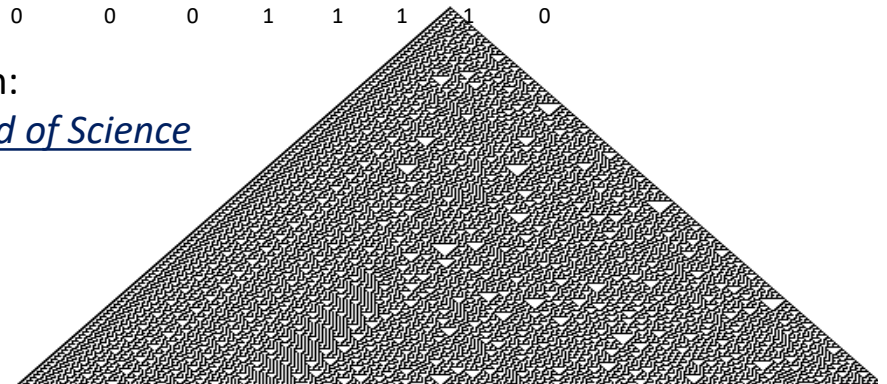
Moore's neighborhoods

Rule 30 cellular automaton

current pattern	111	110	101	100	011	010	001	000
new state for center cell	0	0	0	1	1	1	1	0

S. Wolfram:

[A New Kind of Science](#)



Context for CA within this course

- CA are classical example for complex systems: simple rules give rise to “emergent” behavior.
- CA are actually dynamical systems, with the rules giving the “flow” evolution, with discrete times.
- MD gives evolution of particles with time, continuously in space but with discrete time evolution. CA is often used to model particle evolution, but both space and time are discrete. Using small enough discretization can often mimic MD results, at a fraction of the computational cost.

Some relevant historic landmarks for CA

1940's: von Neumann's cellular automata (inspired by Stan Ulam, both at LANL). Two-dimensional, self-replicating universal copier and constructor. Neighborhood : only those orthogonal cells that touch are neighbors; and with 29 states per cell. Design known as the tessellation model, and called a von Neumann universal constructor.



1970s: Game of Life . Two-state, two-dimensional cellular automaton. Invented by John Conway, popularized by Martin Gardner (*Scientific American*). Rules for cells: 2 black neighbors → stays the same; 3 black neighbors → it becomes black; all other situations it becomes white. Diversity of behavior, fluctuating between apparent randomness and order. Frequent occurrence of *gliders* (cells that move themselves across the grid). It can be arranged that the gliders interact to perform computations: the Game of Life can emulate a universal Turing machine.

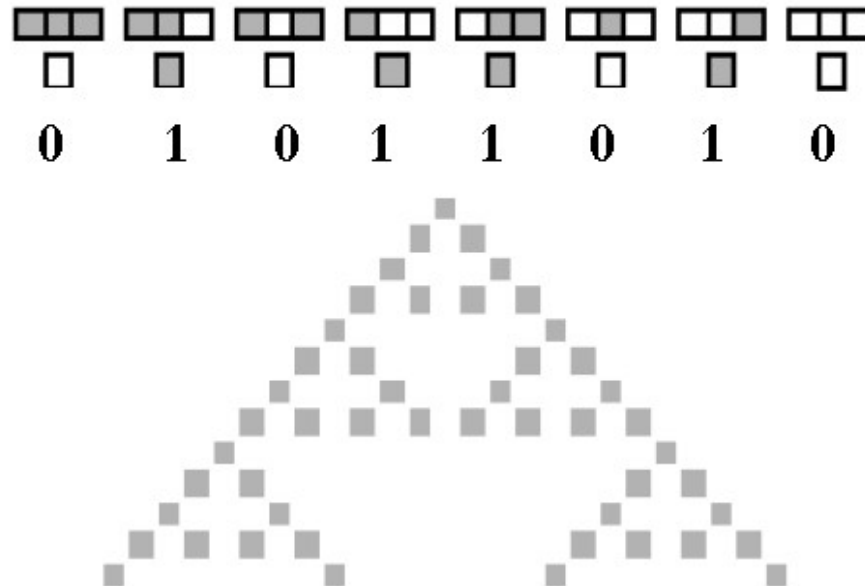
1983: Stephen Wolfram published the first of a series of papers systematically investigating a very basic but essentially unknown class of cellular automata, which he terms elementary cellular automata. Wolfram suspects that complexity in nature may be due to similar mechanisms, and formulated the concepts of intrinsic randomness and computational irreducibility, suggesting that rule 110 may be universal—proved by Matthew Cook in the 1990s.

2002: Wolfram published *A New Kind of Science*, arguing that the discoveries about cellular automata are not isolated facts but are robust and have significance for all disciplines of science.

Elementary cellular automata

- Simplest nontrivial CA: 1D, two possible states per cell, cell's neighbors: adjacent cells.
- Neighborhood of 3 cells $\rightarrow 2^3=8$ possible patterns for a neighborhood.
- Rule for each pattern: whether the cell will be a 1 or a 0 in the next generation $\rightarrow 2^8=256$ rules. Standard naming convention for 256 CAs: [Wolfram code](http://mathworld.wolfram.com/ElementaryCellularAutomaton.html), a number from 0 to 255.

Rule 90



$$90 = (01011010)_{\text{base } 2} = 0 \cdot 2^7 + 1 \cdot 2^6 + 0 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0$$

<http://mathworld.wolfram.com/ElementaryCellularAutomaton.html>

Wolfram's classification

Class 1: Nearly all initial patterns evolve quickly into a stable, homogeneous state. Any randomness in the initial pattern disappears → **Point attractors.**

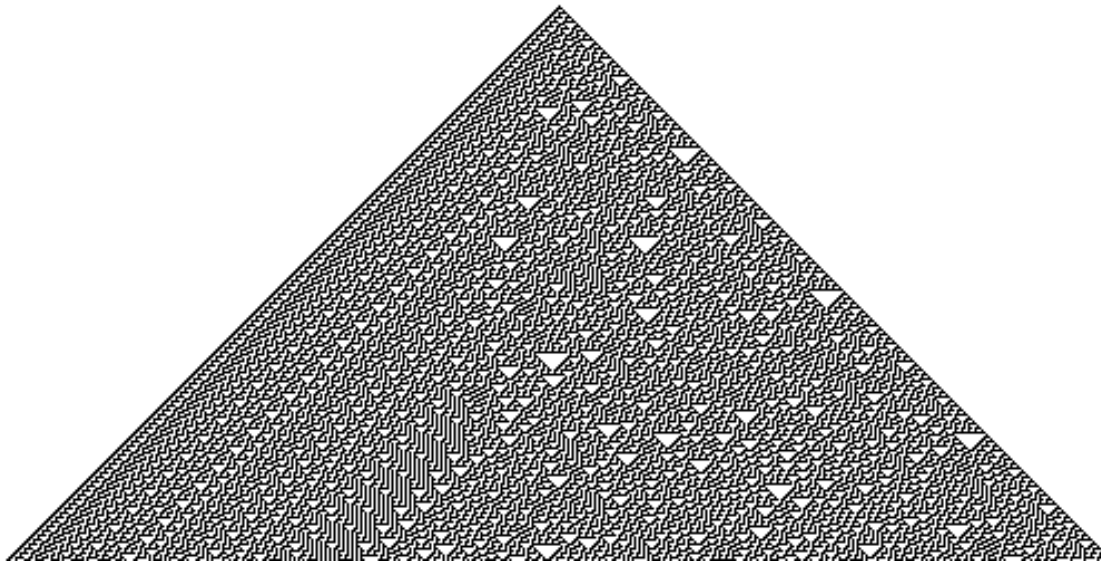
Class 2: Nearly all initial patterns evolve quickly into stable or oscillating structures. Some of the randomness in the initial pattern may filter out, but some remains. Local changes to the initial pattern tend to remain local → **Limit cycles.**

Class 3: Nearly all initial patterns evolve in a pseudo-random or chaotic manner. Any stable structures that appear are quickly destroyed by the surrounding noise. Local changes to the initial pattern tend to spread indefinitely → **Chaotic. (but finite CA has to repeat itself eventually)**

Class 4: Nearly all initial patterns evolve into structures that interact in complex and interesting ways. → **Structured.** Class 2 type stable or oscillating structures may be the eventual outcome, but the number of steps required to reach this state may be very large, even when the initial pattern is relatively simple. Local changes to the initial pattern may spread indefinitely. Wolfram has conjectured that many, if not all class 4 cellular automata are capable of universal computation. This has been proven for Rule 110, and Conway's game of Life.

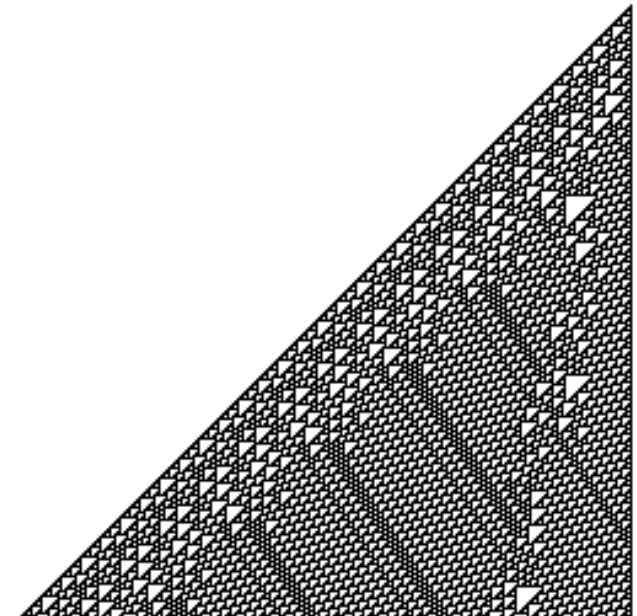
Rule 30 cellular automaton

Starting configuration consists of a 1 (at the top of each image) surrounded by 0's. Each row of pixels represents a generation in the history of the automaton, with $t=0$ being the top row. Each pixel is colored white for 0 and black for 1.



current pattern	111	110	101	100	011	010	001	000
new state for center cell	0	0	0	1	1	1	1	0

Rule 110 cellular automaton



current pattern	111	110	101	100	011	010	001	000
new state for center cell	0	1	1	0	1	1	1	0

Rule 30: *class 3* behavior: simple input patterns can lead to chaotic, seemingly random histories.

Rule 110: *class 4* behavior, which is neither completely random nor completely repetitive. It presents universality. Localized structures appear and interact in various complicated-looking ways. It has been the basis over which some of the smallest universal Turing machines have been built, inspired on the breakthrough concepts that the development of the proof of rule 110 universality produced.

Game of Life (GoL) ... probably most popular 2D CA

2 possible states per cell, NxM grid
Moore neighborhood

<http://golly.sourceforge.net>, for Linux/Windows/Mac (2022), Android (2018)
<https://golly.sourceforge.io/webapp/golly.html> (2022)

- If an alive cell has fewer than 2 alive neighbors, it dies (**loneliness**)
- If an alive cell has more than 3 alive neighbors, it dies (**overcrowding**)
- If an alive cell has either 2 or 3 alive neighbors, it goes on living (**happiness**)
- If a dead cell has exactly 3 alive neighbors, it comes alive (**reproduction**).
- **Otherwise it stays** dead.

A puffer-type breeder
(red) that leaves glider
guns (green) in its wake,
which in turn create
gliders (blue) → **Emergent**



Generalizations of the CA concept.

- Use Genetic Algorithms to find rules which can lead to certain computations (see work by M. Mitchell et al., on emergent computation).
- Use grids other than rectangular/cubic: triangular, hexagonal, etc. They are still space oriented, not agent-based
- Rules can be probabilistic rather than deterministic. A probabilistic rule gives, for each pattern at time t , the probabilities that the central cell will transition to each possible state at time $t+1$. Sometimes a simpler rule is used; for example: "The rule is the Game of Life, but on each time step there is a 0.001% probability that each cell will transition to the opposite color."
- The neighborhood or rules could change over time or space. For example, initially the new state of a cell could be determined by the horizontally adjacent cells, but for the next generation the vertical cells would be used.
- The grid can be finite, so that patterns can "fall off" the edge of the universe.
- In CA, the new state of a cell is not affected by the new state of other cells. This could be changed so that, for instance, a 2 by 2 block of cells can be determined by itself and the cells adjacent to itself.
- Continuous automata. Instead of the rule and states being discrete (e.g. a table, using states $\{0,1,2\}$), continuous functions are used, and the states become continuous (usually values in $[0,1]$). The state of a location is a finite number of real numbers. Certain CA can yield diffusion in liquid patterns in this way. Continuous spatial automata have a continuum of locations. The state of a location is a finite number of real numbers. Time is also continuous, and the state evolves according to differential equations. One important example is reaction-diffusion textures, differential equations proposed by Alan Turing to explain how chemical reactions could create the stripes on zebras and spots on leopards. When these are approximated by CA, such CAs often yield similar patterns. There are known examples of continuous spatial automata which exhibit propagating phenomena analogous to gliders in the Game of Life.

There are many CA applications

Biology

Pattern formation: seashells, chromatophores in a cephalopod skin. Leaf stoma, neurons, flocking, etc.



Chemistry

Reaction-diffusion equations: **self-assembly**

Physics

Granular systems

Mathematics

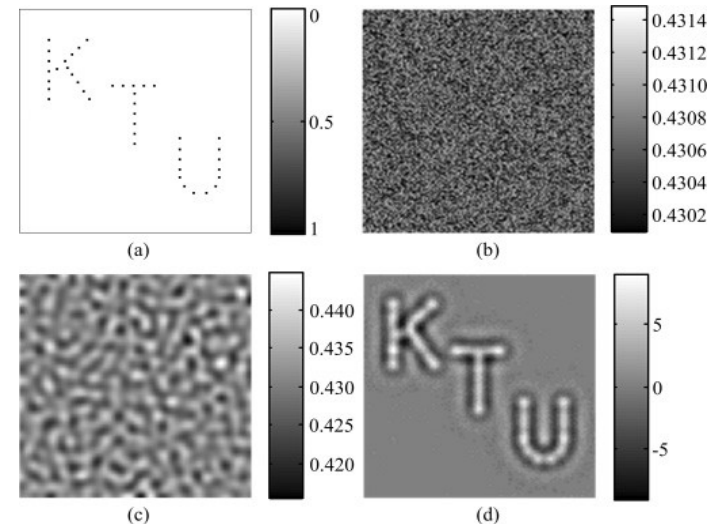
Numerical Differential Equation solvers

Computer Science

Turing machine, cryptography, prime number generation, gaming (***Minecraft***), etc.

Miscellanea

Stock market, car traffic flow, fire propagation (Printista *et al.*, UNSL), crowd behavior, etc.



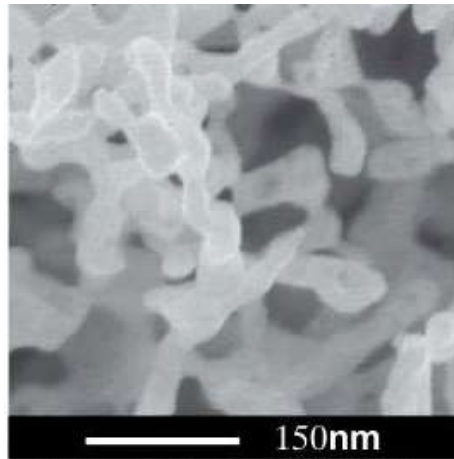
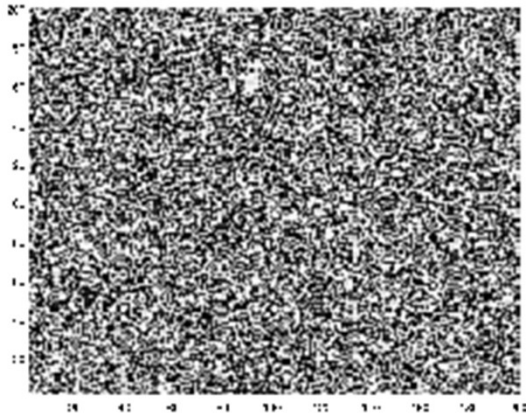
PRE **84**, 056213 (2011)

“Reaction-diffusion equation” to model foams

D. Schwen, A. Caro (LANL), D. Farkas (Va Tech)

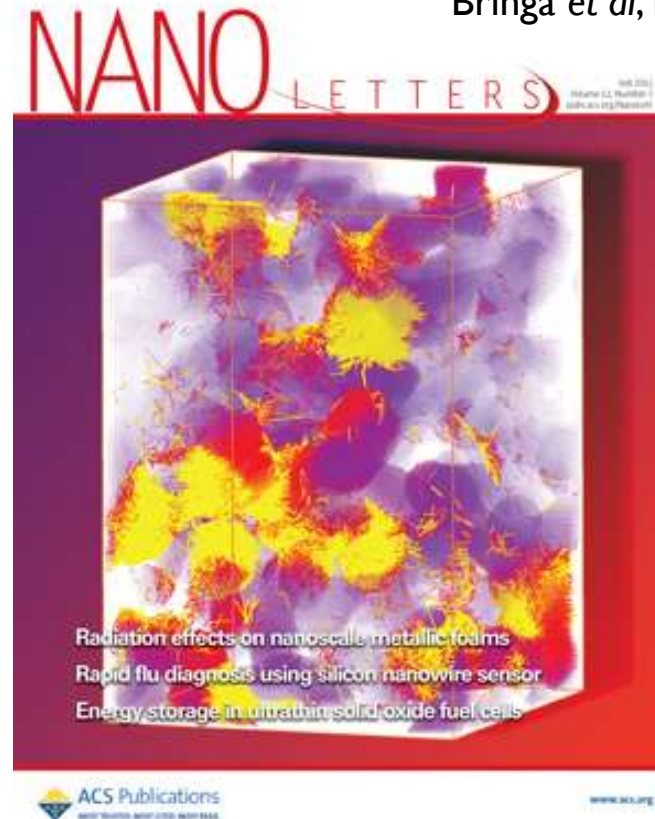
http://en.wikipedia.org/wiki/Spinodal_decomposition

Uses Cahn-Hilliard Equation, to generate 3D foam. OpenCL code by Schwen needs modifications for future research



Plasma exposed W-C surface
Takamura et al., Plasma and
Fusion Research 1, 51 (2006)

Bringa et al, NanoLetters (2012)



Outline

- CA: Definition, context and history.
- 1D CA and their classification
- 2D CA: Game of Life (GoL)
- Generalization and applications of CA
- Computational issues, demonstrated by two examples: GoL and settlement dynamics.
- Software for CA simulations: focus on *NetLogo* and additional examples.
- Bibliography: articles and web sites
- Computational Lab

HPC for CA in CPU+GPU, using OMP, MPI y CUDA

Flexible CA in parallel arch using:

- ✓ OpenMP for multicore computers
- ✓ MPI for CPU clusters
- ✓ CUDA for GPUs
- ✓ CUDA + MPI for GPU clusters.



Paradigm: Game of Life (GoL)
2 states, NxM grid
Moore neighborhood

Code: https://sites.google.com/site/simafweb/proyectos/ca_gpu

Details in two publications (Record: up to 16384^2 cells ~ 268.4e6 cells):

Millán, E.N., Wolovick, N., Piccoli, M.F. *et al.*, *Performance analysis and comparison of cellular automata GPU implementations*. Cluster Computing **20**, 2763–2777 (2017).
<https://doi.org/10.1007/s10586-017-0850-3> (12 citations)

Millán, E.N., Bederian, C., N., Piccoli, M.F. *et al.*, *Performance analysis of cellular automata HPC implementations*. Computers & Electrical Engineering **48**, 12-24 (2015).
<http://dx.doi.org/10.1016/j.compeleceng.2015.09.015> (13 citations)

New review (up to 16384^2 cells ~ 268.4e6 cells, same as before):

D. Cagigas Muñoz *et al.*, *Efficient simulation execution of cellular automata on GPU*, Simulation Modelling Practice and Theory **118**, 102519 (2022)

Baseline GoL implementations

Serial

- ✓ 2 Grids: main and secondary
- ✓ For each cell of main: apply rules + save on secondary
- ✓ Copy secondary grid into main grid

MPI

- ✓ Domain decomposition: each process solves a block.
- ✓ Exchange borders amongst neighbor blocks.

GPU

- 2 kernels
- ✓ K1: System Evolution,
- ✓ K2: Update state for next iteration.

GPU+MPI

- Evolution entirely in GPU
- ✓ Each MPI process updates block boundaries

Hardware:

Cluster ICB-ITIC (128 CPU cores -AMD-, 1 Tesla C2050, UN Cuyo).

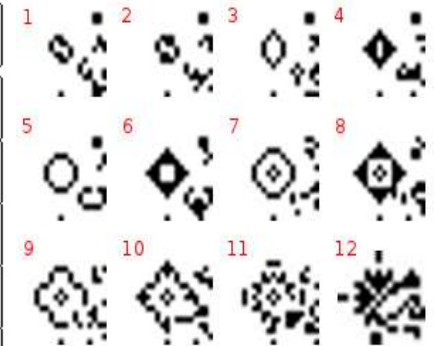
Cluster Mendieta (128 cores -Intel-, 12 Tesla M2090, UN Córdoba).

Results for Mendieta Cluster (1e3 iterations)

Preliminary timing (CACIC2013). Grid NxN
Significantly improved later on

N	CPU Serial				OpenMP				MPI			
	fill	evolve	output	total	fill	evolve	output	total	fill	evolve	output	total
500	0.03	2.21	0.04	2.25	0.005	1.30	0.06	1.37	0.006	0.41	0.1	0.52
1000	0.01	8.94	0.16	9.11	0.01	5.25	1.07	6.33	0.03	1.22	0.41	1.66
2000	0.04	40.27	0.62	40.93	0.04	25.33	1.05	26.41	0.14	4.35	1.54	6.03
4000	0.16	188.49	3.78	192.42	0.27	101.05	4.23	105.54	0.59	26.66	7.01	34.26
6000	0.34	376.71	5.58	382.64	0.35	225.56	9.66	235.57	1.32	59.59	13.86	74.76

N	GPU				Multi-GPU				
	fill	evolve	output	total	fill	evolve	output	halo	total
500	3.78	0.13	1.15	5.06	3.99	0.51	0.19	0.43	5.11
1000	3.81	0.43	0.31	4.55	4.08	0.53	0.42	0.55	5.59
2000	3.9	1.64	0.8	6.34	4.06	0.75	1.73	0.8	7.34
4000	4.09	5.66	2.98	12.73	4.3	1.32	6.77	1.28	13.67
6000	4.21	14.36	13.35	31.92	4.7	3.43	15.05	3.21	26.39
6000	running in 2 GPU per node				1.48	3.29	14.31	0.18	19.27



Halo:
communication time

Results: Speedups for grid dimxdim

Workstation Phenom, Tesla C2050

dim	Speedup vs Serial Code			
	MPI	OMP	GPU	Multi-GPU
500	8.32	2.33	8	0.97
1000	7.44	2.8	11.92	3.49
2000	4.62	2.64	14.91	8.94
4000	5.01	2.84	15.72	14.73
6000	5.07	2.88	14.53	17.46
AVG	6.09	2.70	13.02	9.12

Cluster Mendieta, Tesla M2090

dim	Speedup vs Serial Code			
	MPI	OMP	GPU	Multi-GPU
500	4.33	1.64	0.45	0.44
1000	5.50	1.44	2.00	1.63
2000	6.79	1.55	6.46	5.58
4000	5.62	1.82	15.11	14.08
6000	5.12	1.62	11.99	14.50
6000 in 2 GPUs per node				19.86
AVG	5.47	1.62	7.20	9.35

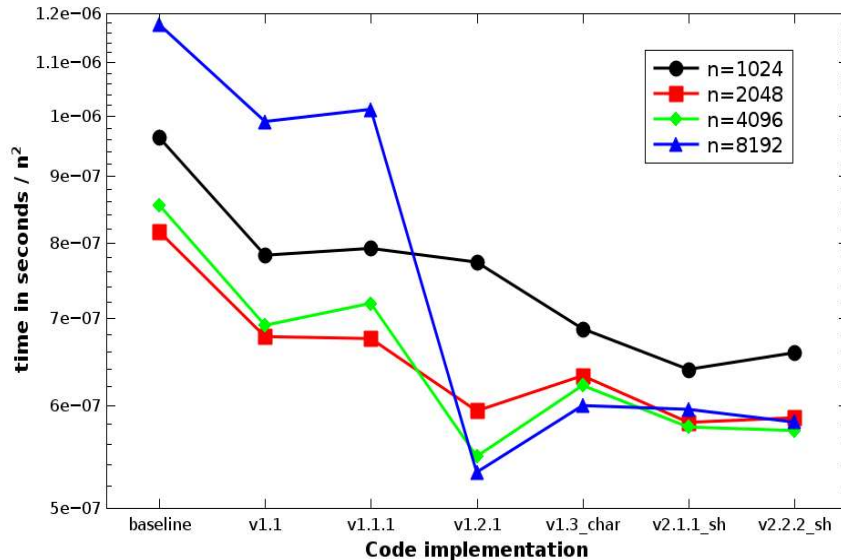
Millán *et al.*, CACIC 2013

Can these baseline results be improved?

You bet!

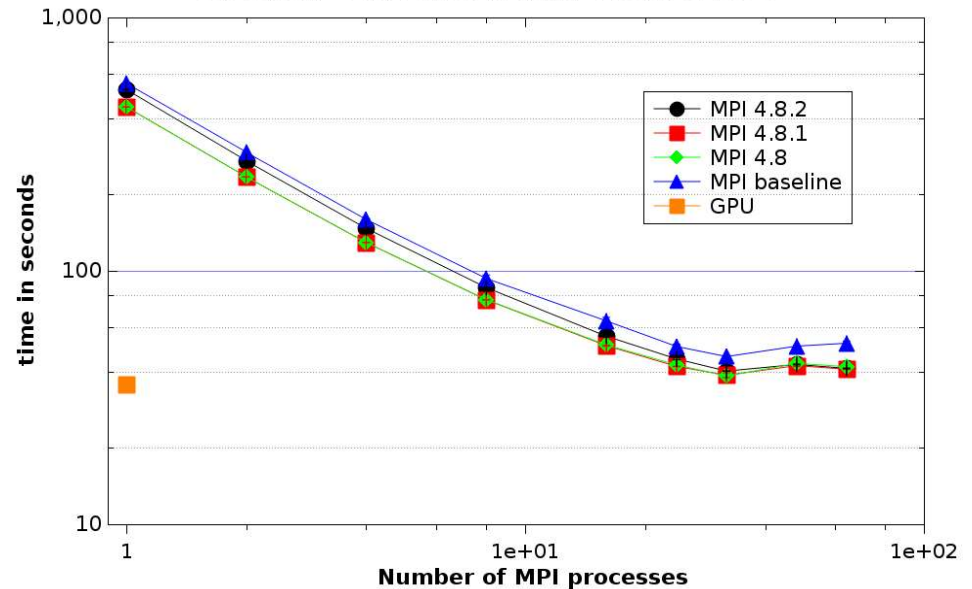
Results: Code improvements + Scaling

Performance of GPU implementations
in NVIDIA Tesla c2050 GPU

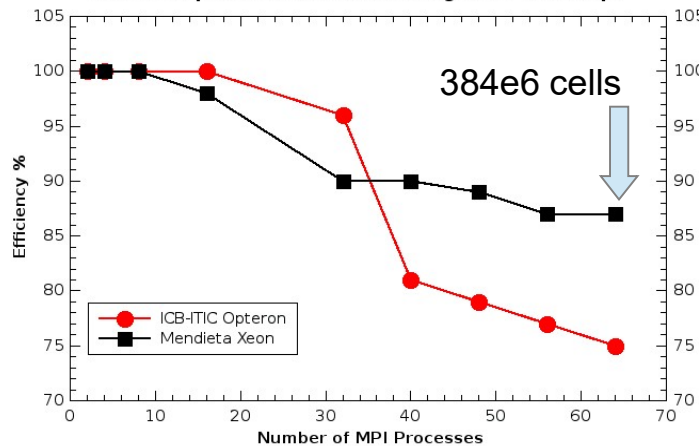


Cluster TOKO (ICB-ITIC)

MPI vs best GPU implementation
Strong scaling with $n=8192$
Opteron 6272 and Nvidia Tesla C2050

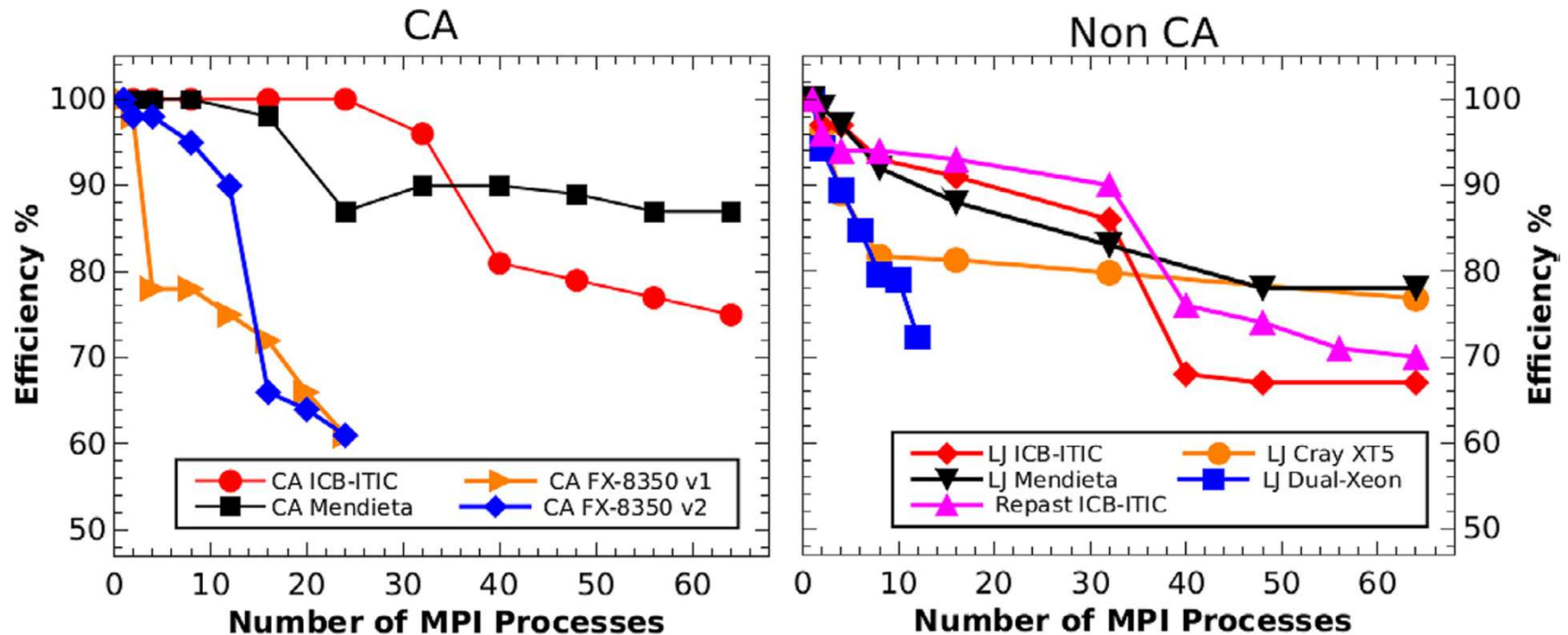


Efficiency Weak Scaling
Evolve time: communication time + step time - output time
Each core processes a 3000x2000 grid for 500 steps



MPI: AMD vs Intel issues
GPU+MPI: not very good ..

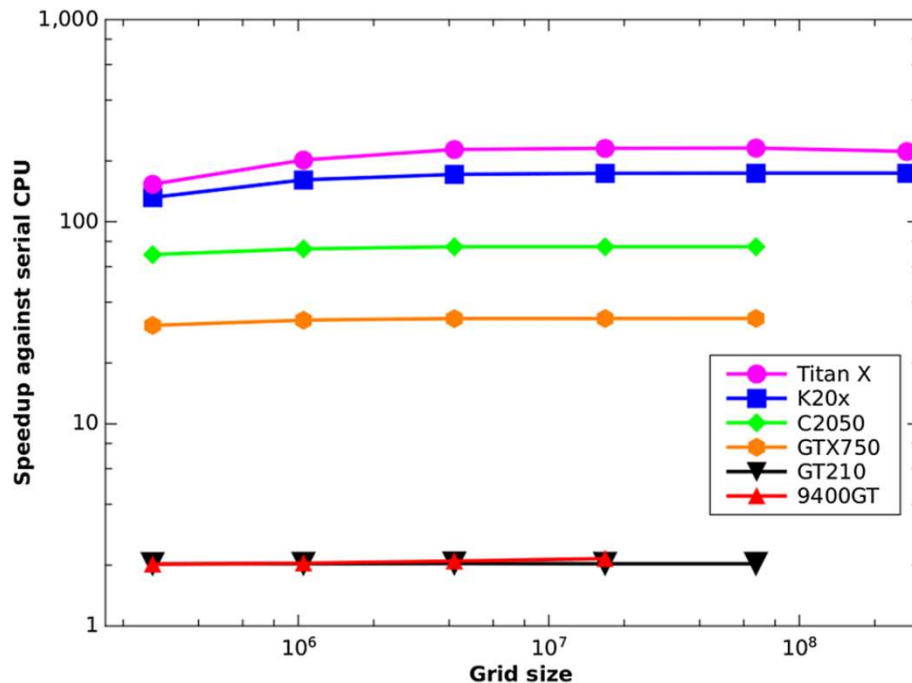
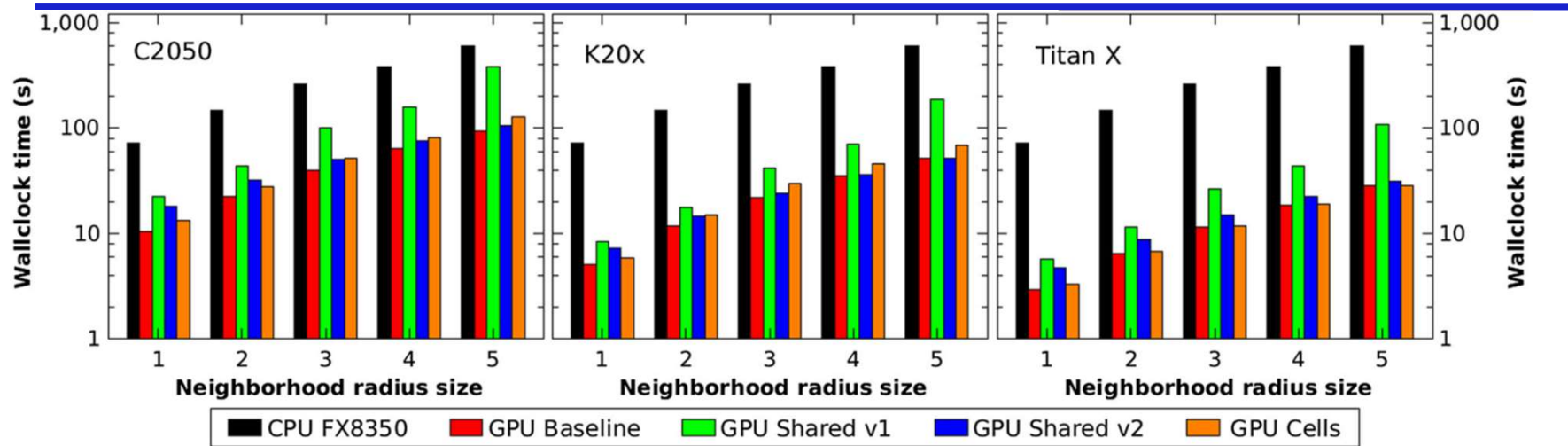
Weak scaling of CA CPU code (grid 3000x2000)



Promising results:

Parallel domain decomposition approach is reasonably efficient, given that compares well with other mature parallel codes (LAMMPS for LJ and REPAST for Zombie ABM).

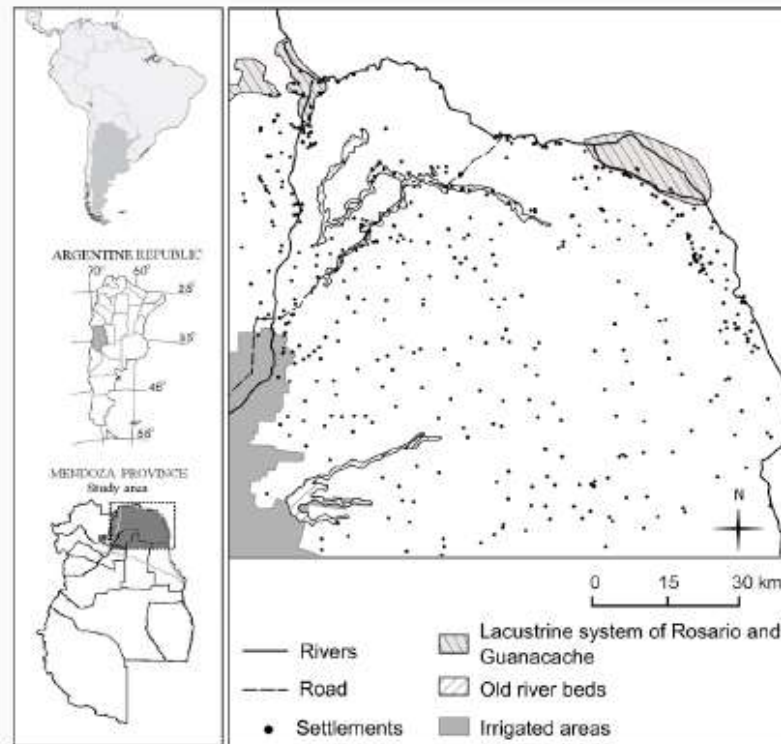
GPU code flexibility



Code can deal with large neighborhoods: Different GPU implementations with different neighborhood radius, executed in the C2050, K20x and Titan X GPUs, for a lattice size of 4096×4096 during 1000 steps

Speed-up improves for more complex evolution (cosine evaluation): CA simulations for different lattice sizes, with the best implementation for each GPU against an optimized Serial CPU implementation

CA- Settlement Dynamics



Challenge:

can we predict with a CA the settlement evolution, starting from a few settlement decades ago, and using rules based on environmental factors like water proximity? No ... need to add some stochastic contributions with Monte Carlo (social factors difficult to translate into CA rules?)

CA- Settlement Dynamics

Factores que determinan el uso del espacio y los recursos en zonas áridas no irrigadas del noreste de Mendoza (Argentina).

S.B. Goirán *et al.*, Servicios ecosistémicos hídricos: estudios de caso en América Latina y el Caribe. Lara A, Laterra P, Manson R, Barrantes G (eds). Red ProAgua CYTED. Imprenta América. Valdivia, Chile 312 pp. 2013

Monte Carlo simulations of settlement dynamics in GPUs,

E. Millan *et al.*, Cluster Computing **19**, 557 (2016)

Monte Carlo model framework to simulate settlement dynamics

E. Millan *et al.*, Ecological Informatics **36**, 135 (2016).

Livestock Settlement Dynamics in Drylands: Model application in the Monte desert (Mendoza, Argentina),

E. Millan *et al.*, Ecological Informatics **39**, 84 (2017).

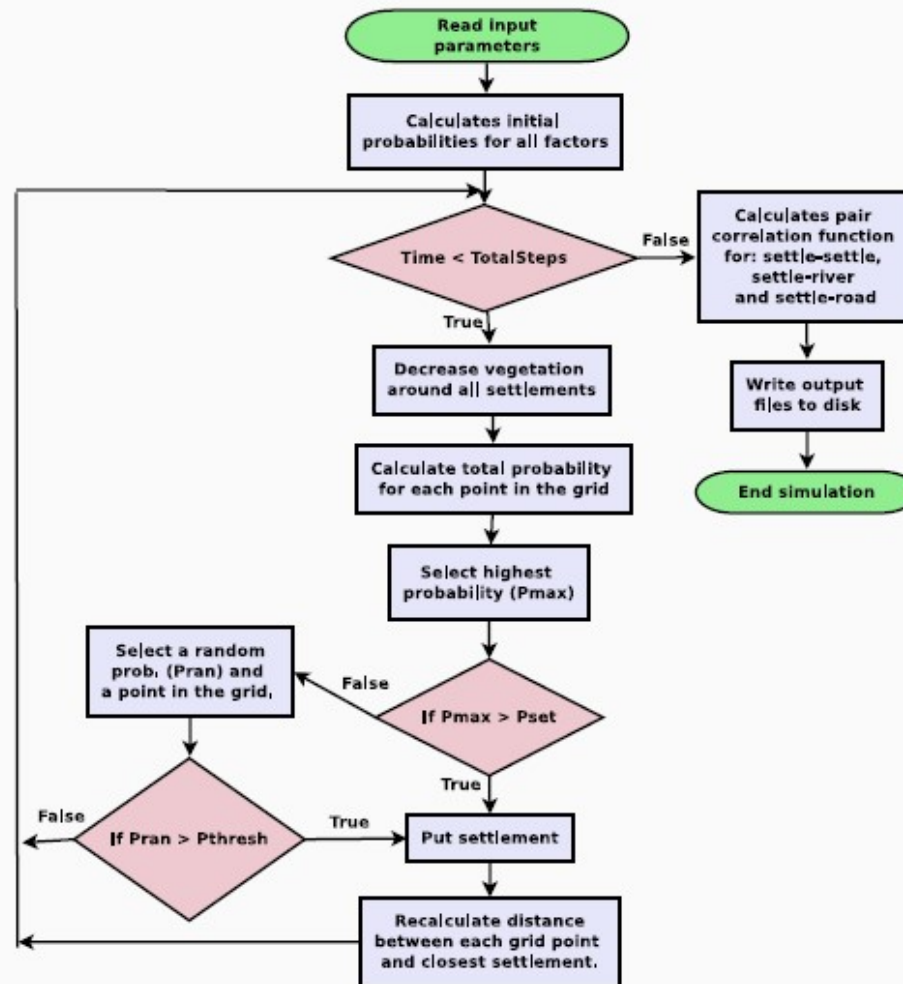
New related paper:

Modeling habitat suitability and spread dynamics of two invasive rose species in protected areas of Mendoza, Argentina, A. Mazzolari *et al.*, Ecological Complexity **44, 100868 (2020).**

CA- Settlement Dynamics

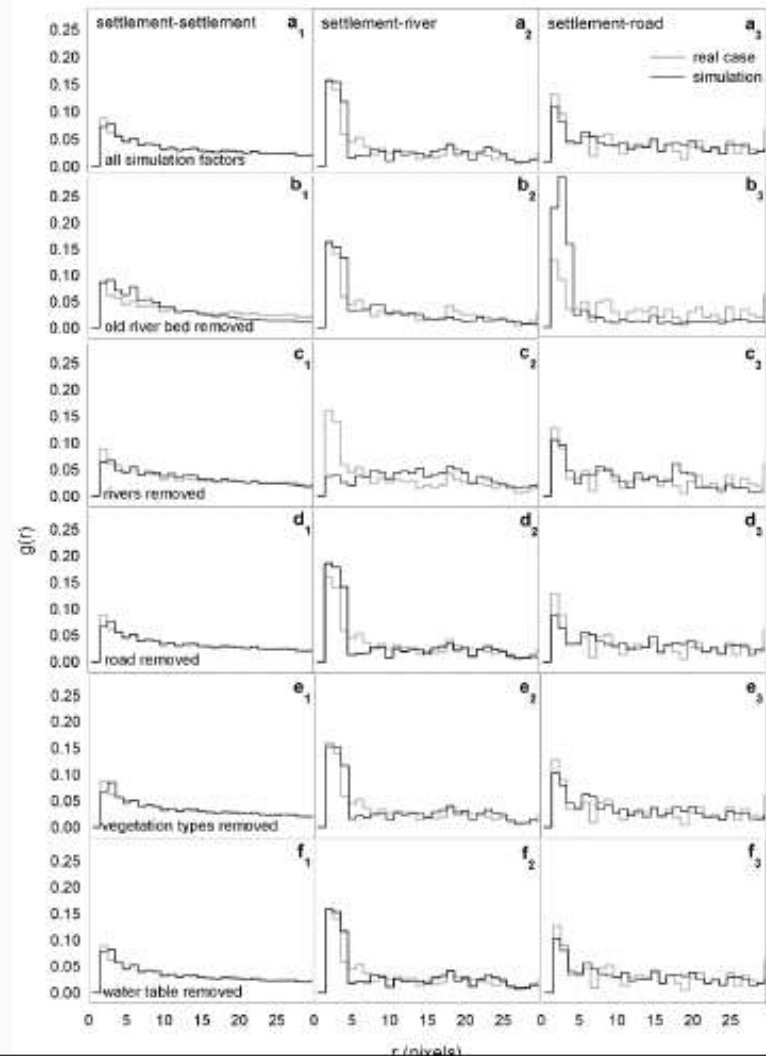
- Se desarrolló un framework basado en el modelo de CA con una componente del modelo de Monte Carlo para la simulación de dinámica de asentamientos.
 - El framework utiliza factores ambientales para realizar la evolución del sistema.
 - Monte Carlo se utiliza para modelar factores sociales no incluidos en los factores ambientales.
 - Se desarrolló el framework en CPU serial, OpenMP y GPUs.
 - Una comprensión de la relación entre los recursos naturales y los asentamientos en el presente es crucial para estimar futuros efectos de los cambios ambientales y socio-económicos en la región simulada.
1. **Etap Input:** se leen parámetros y mapas de cada factor.
 2. **Etap Setup:** se calcula la distancia entre cada celda y la ruta, río y asentamiento más cercano.
 3. **Etap Evolve:** se calculan probabilidades y crean asentamientos, mientras *paso* < *pasos_totales*.
 4. **Etap Output:** se escriben en disco mapas y correlación de pares.

CA- Settlement Dynamics

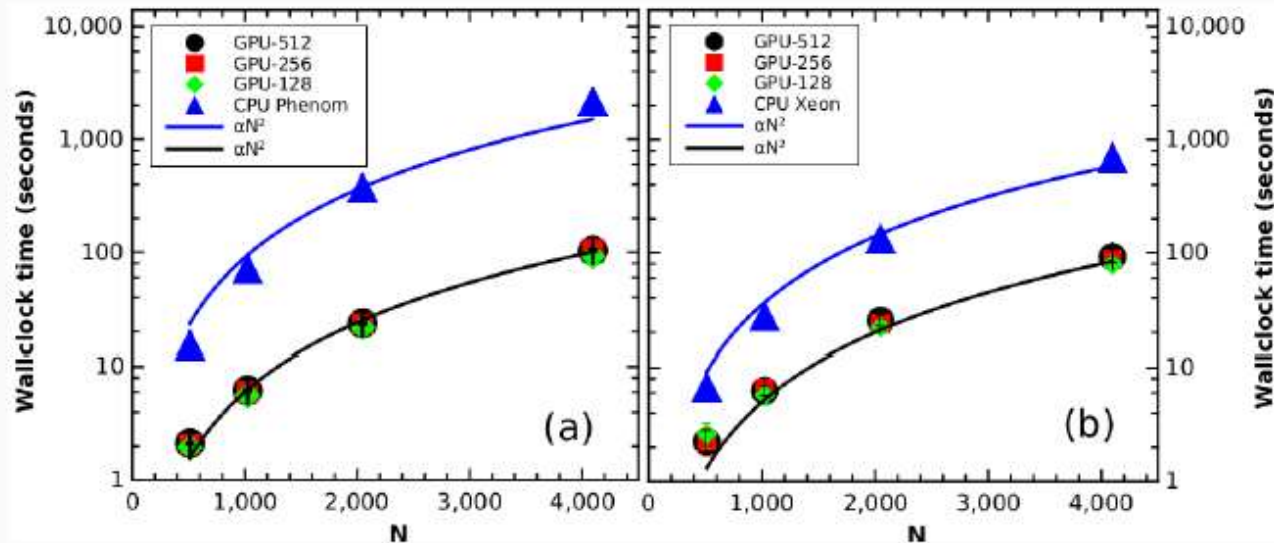


CA- Settlement Dynamics: spatial correlation functions

Histogramas de correlación de pares $g(r)$ para distancia entre asentamientos (1-Settle-Settle), asentamiento a río (2-Settle-River), y asentamiento a ruta (3-Settle-Road); (a) Simulaciones con todos los factores; (b) sin paleocanal (old river bed); (c) sin río; (d) sin ruta; (e) sin tipos de vegetación; (f) sin agua subterránea. r (pixels) es la distancia en pixels, donde un pixel = 0.75 km de lado.

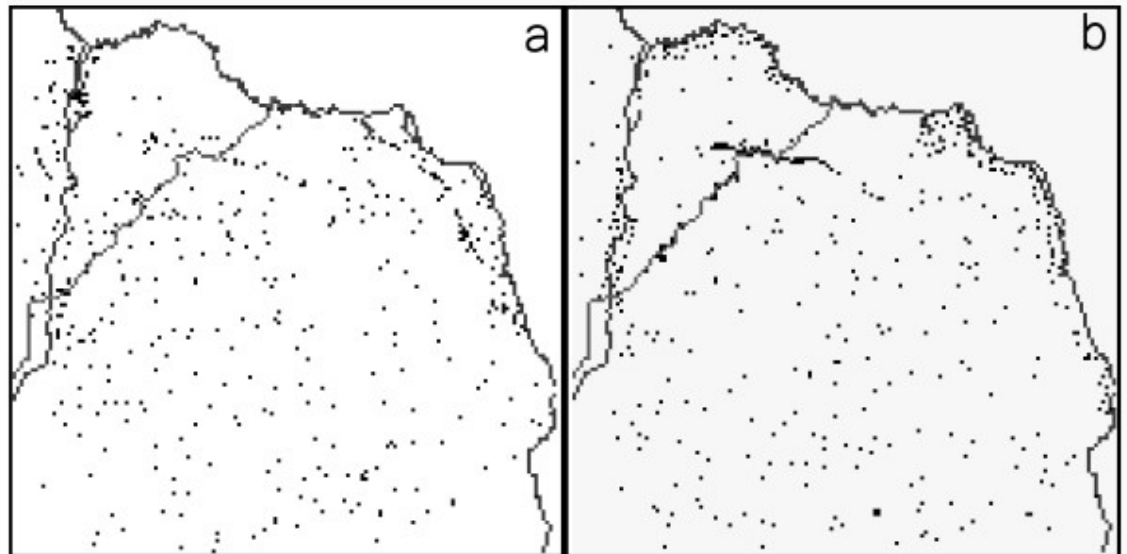


CA- Settlement Dynamics: computational scaling



Challenge:

Need statistics ...
together with some
measurement of
convergence to an
“equilibrium” or “steady”
state.



Outline

- CA: Definition, context and history.
- 1D CA and their classification
- 2D CA: Game of Life (GoL)
- Generalization and applications of CA
- Computational issues, demonstrated by two examples: GoL and settlement dynamics.
- Software for CA simulations: focus on *NetLogo* and additional examples.
- Bibliography: articles and web sites
- Computational Lab

Software (most ABM codes run CA)

http://en.wikipedia.org/wiki/Comparison_of_agent-based_modeling_software

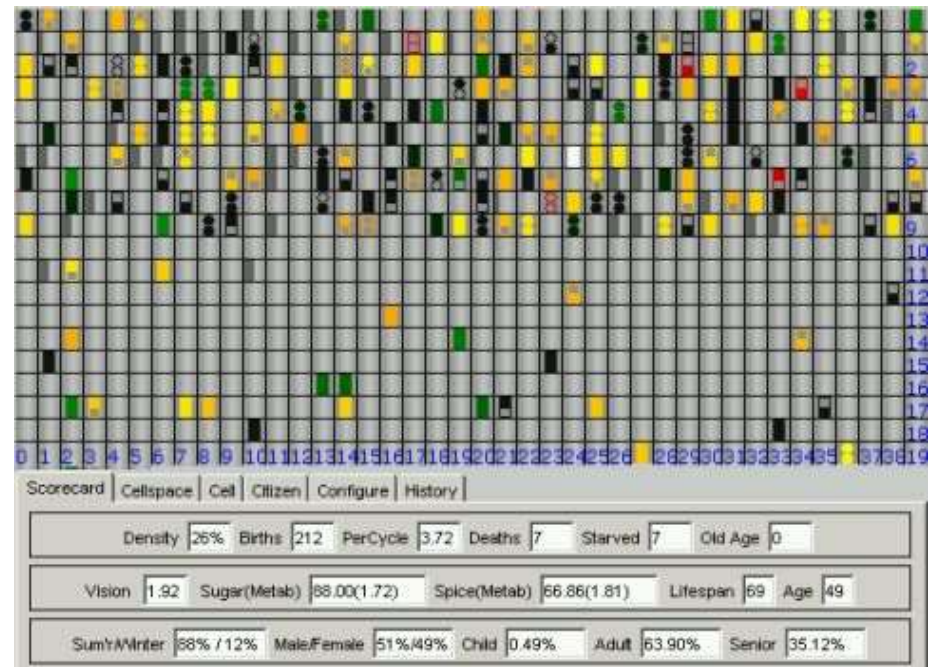
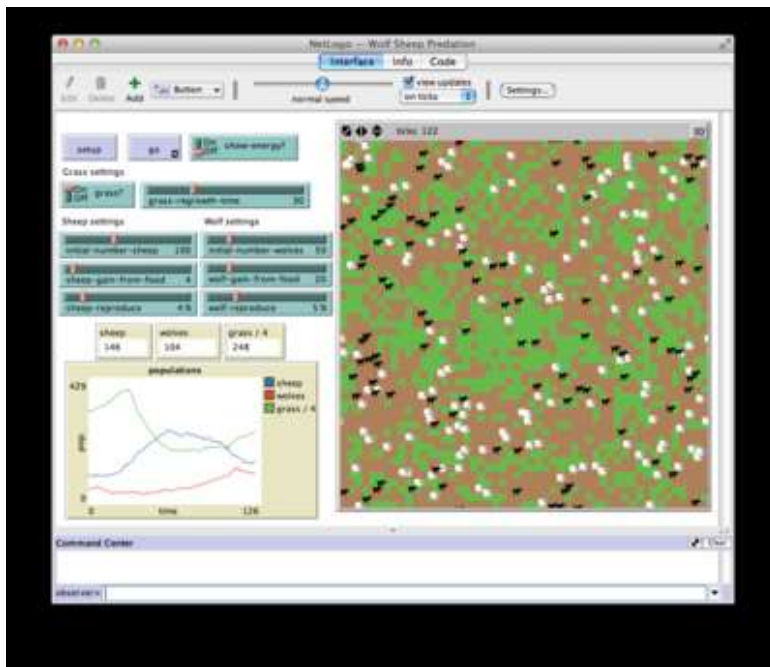
https://es.wikipedia.org/wiki/Comparaci%C3%B3n_de_software_de_modelaje_basado_en_agentes

<https://www.comses.net/resources/modeling-platforms/>

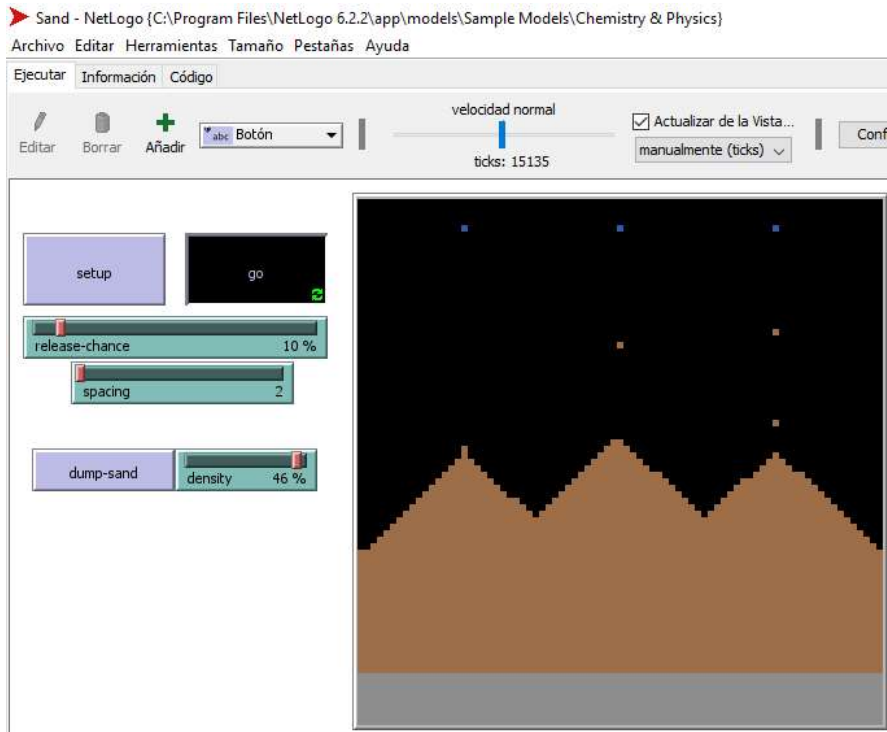
Netlogo <http://ccl.northwestern.edu/netlogo/models/community/> (GIS)

RePast/Sugarscape *REcursive Porous Agent Simulation Toolkit*. GIS compatible

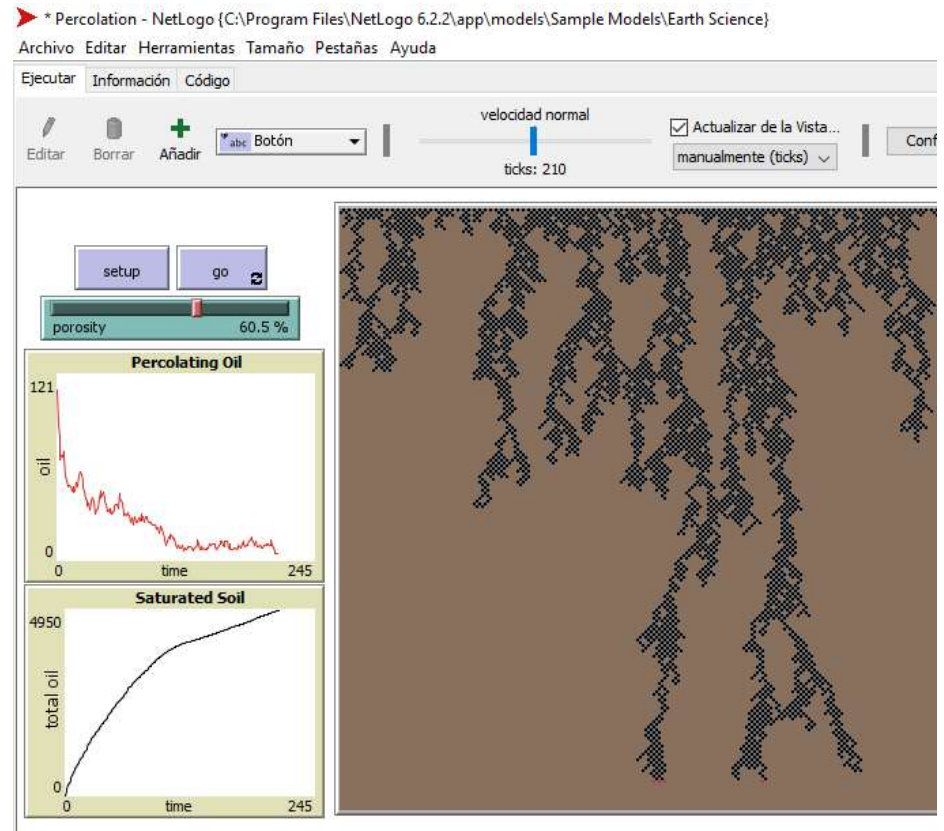
<http://repast.sourceforge.net/> <http://sugarscape.sourceforge.net/>



Some more examples from *NetLogo*

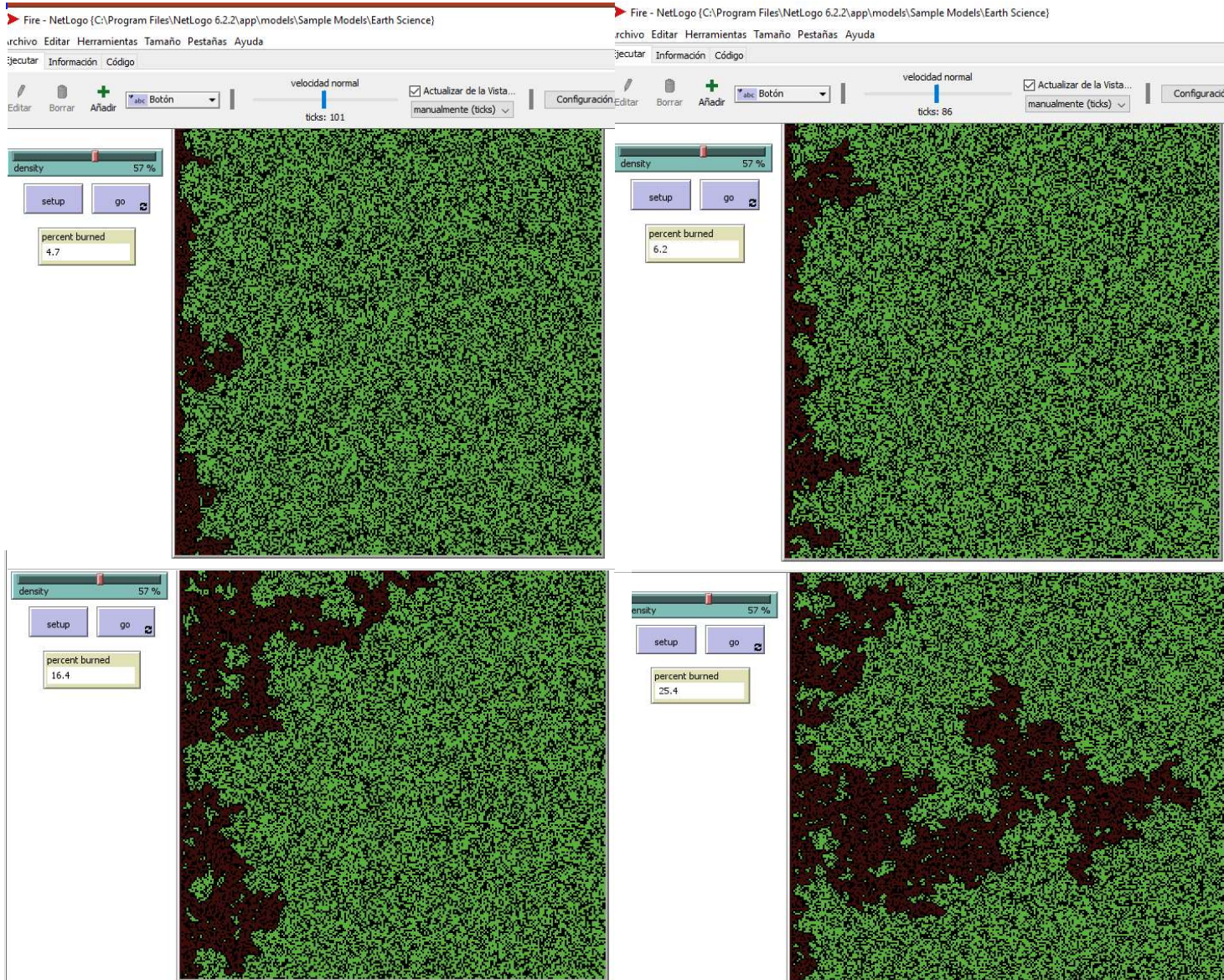


CA rules: cell changes state
(occupation by sand grain) based on
neighbor state.



CA rules: cell changes state (oil or
empty) based on soil porosity

Another example from *NetLogo*: need for statistics



CA for FIRE spread:
Percentage burn depends on tree density, but there is a large variation near the percolation threshold (57% tree density can reach percolation)

Bibliography (if you want deeper understanding and fun examples)

General references:

- Stat Mech of CA, S. Wolfram, Rev. Mod. Phys. (1983):
<https://www-users.york.ac.uk/~gt512/BIC/wolfram83.pdf>
- CA as models of complexity, S. Wolfram, Nature (1984):
https://fricke.co.uk/Teaching/CS523_2017spring/Readings/Wolfram1984.pdf
- Mathematical discussion on CA complexity and chaos: <https://arxiv.org/pdf/2108.00415.pdf>
- Excellent review on 1D CA: Progress, gaps and obstacles in the classification of cellular automata:
<https://doi.org/10.1016/j.physd.2021.133074>
- Cellular Automata, PDEs, and Pattern Formation, <https://arxiv.org/ftp/arxiv/papers/1003/1003.1983.pdf>
- Coarse-graining of cellular automata, emergence, and the predictability of complex systems,
<https://arxiv.org/pdf/nlin/0508033.pdf>

Urban and landscape

The simulation and prediction of spatio-temporal urban growth trends using cellular automata models: A review, <https://doi.org/10.1016/j.jag.2016.07.007> (available from Scholar)

Application of Cellular automata and Markov-chain model in geospatial environmental modeling- A review, <https://doi.org/10.1016/j.rsase.2017.01.005> (available from Scholar)

A review of assessment methods for cellular automata models of land-use change and urban growth, <https://doi.org/10.1080/13658816.2019.1684499>

Cellular Automata Modeling for Urban and Regional Planning,
https://link.springer.com/chapter/10.1007/978-981-15-8983-6_45

Urban Expansion Simulated by Integrated Cellular Automata and Agent-Based Models; An Example of Tallinn, Estonia, <https://www.mdpi.com/2413-8851/5/4/85/pdf>

Bibliography (continuation)

Traffic

Cellular automata models for signalised and unsignalised intersections with special attention to mixed traffic flow: a review, <https://ietresearch.onlinelibrary.wiley.com/doi/pdfdirect/10.1049/iet-its.2020.0062>

Review of the cellular automata models for reproducing synchronized traffic flow,
<https://doi.org/10.1080/23249935.2020.1810820>

Evacuation

A review of cellular automata models for crowd evacuation, <https://doi.org/10.1016/j.physa.2019.03.117>
(available from Scholar)

A continuous floor field cellular automata model with interaction area for crowd evacuation:
<https://doi.org/10.1016/j.physa.2021.126049> (available from Scholar)

A cellular automata model for high-density crowd evacuation using triangle grids:
<https://doi.org/10.1016/j.physa.2018.06.055>

Research and realization of parallel algorithms for large scale crowd evacuation in emergency:
<https://doi.org/10.1016/j.matcom.2021.10.026>

ML

Cellular automata as convolutional neural networks: "... any CA may readily be represented using a convolutional neural network with a network-in-network architecture" <https://arxiv.org/pdf/1809.02942.pdf>
<https://github.com/williamgilpin/convoca>

Basic Science

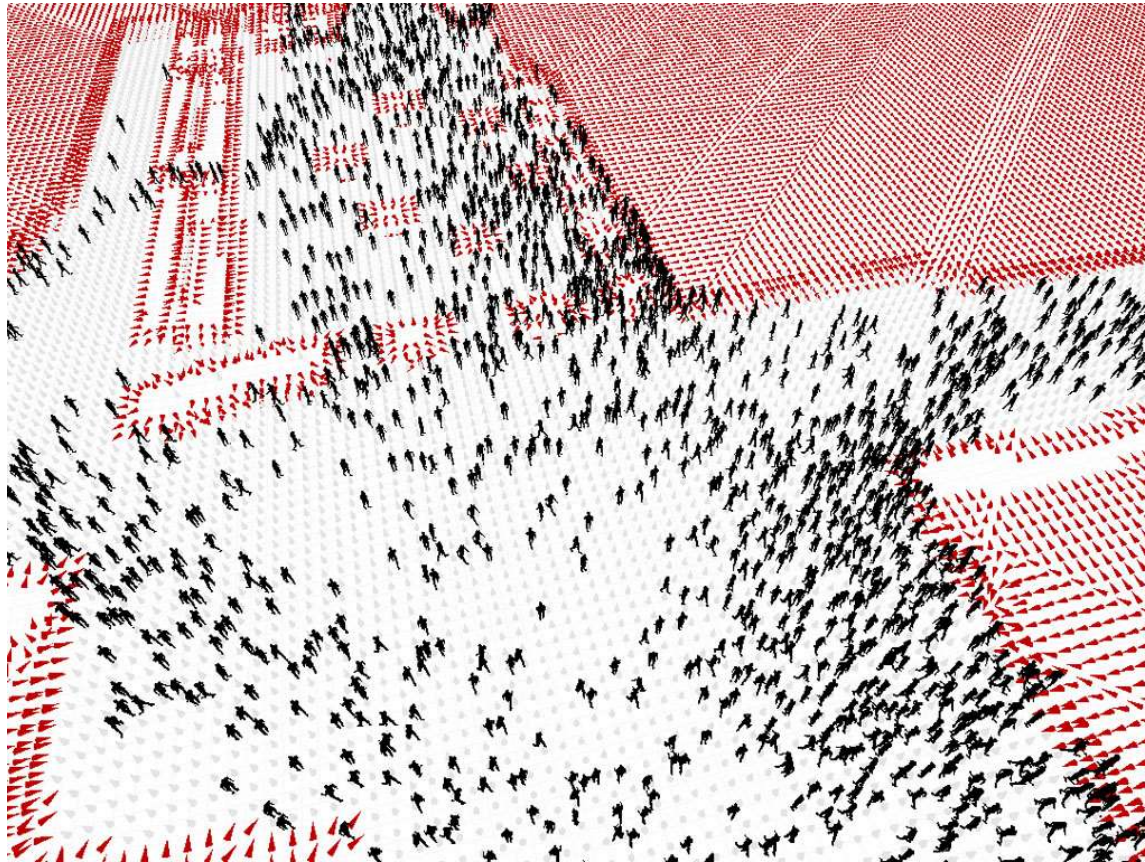
Cellular Automata in Chemistry and Chemical Engineering, Annual Review of Chemical and Biomolecular Engineering 11, 87-108 (2020). <https://doi.org/10.1146/annurev-chembioeng-093019-075250>

Other

"Cellular Automata for the Observation of Complex Systems", doctoral theses by Theo Plénet (02/2023):
https://theses.hal.science/tel-04002482/file/These_Plenet_Theo_2022.pdf

Beyond CA: Agent Based Modeling (ABM)

Flexible Agent Based Simulation for Pedestrian Modeling on GPUs



Richmond Paul, Coakley Simon, Romano Daniela (2009), "Cellular Level Agent Based Modelling on the Graphics Processing Unit", Proc. of HiBi09 - High Performance Computational Systems Biology, 14-16 October 2009, Trento, Italy

<https://flamegpu.com/>

**Several ABMs for CPU clusters, but
there are very few multi-GPU ABMs that I know of**

Agent-Based Modeling (ABM)

http://en.wikipedia.org/wiki/Agent-based_model

□ **What is an agent?**

- A discrete entity with its own goals and behaviors, i.e. certain characteristics and attributes.
- Autonomous, with a capability to adapt and modify its behaviors

□ **Assumptions**

- Some key aspect of behaviors and interactions can be described by a set of rules. These agents need not be rational (letters, many car drivers), but need to have some explicit set of actions which will follow (perhaps according to some probability distribution) from their current state (environment) and the state of other agents
- Complex social processes and a system can be built “from the bottom up.”

□ **Examples**

- People, groups, organizations
- Animal or plants
- Social insects, swarms
- Robots, systems of collaborating robots
- Cars or airplanes.

□ **Agents are diverse and heterogeneous → Rich behavior!**

Grimm et al., Pattern-oriented modeling of agent-based complex systems. Science **310**, 987 (2005).

Agent-Based Modeling (ABM)

- Off-shoot of complexity theory which explores 'complex' systems; the whole is greater than the sum of the parts
- Agent-based simulation can capture "real life" economic/business systems on a computer by replicating the behaviors of **heterogeneous** participants and modeling the **interactions** between them.
- By simulating the nature of the interactions between the agents in a system, ABMs can become a learning tool for understanding a system under a variety of conditions. They are particularly useful for **evolving/dynamic** systems.
- Like other types of simulation techniques, they are meant to complement and enhance rather than supplant traditional approaches

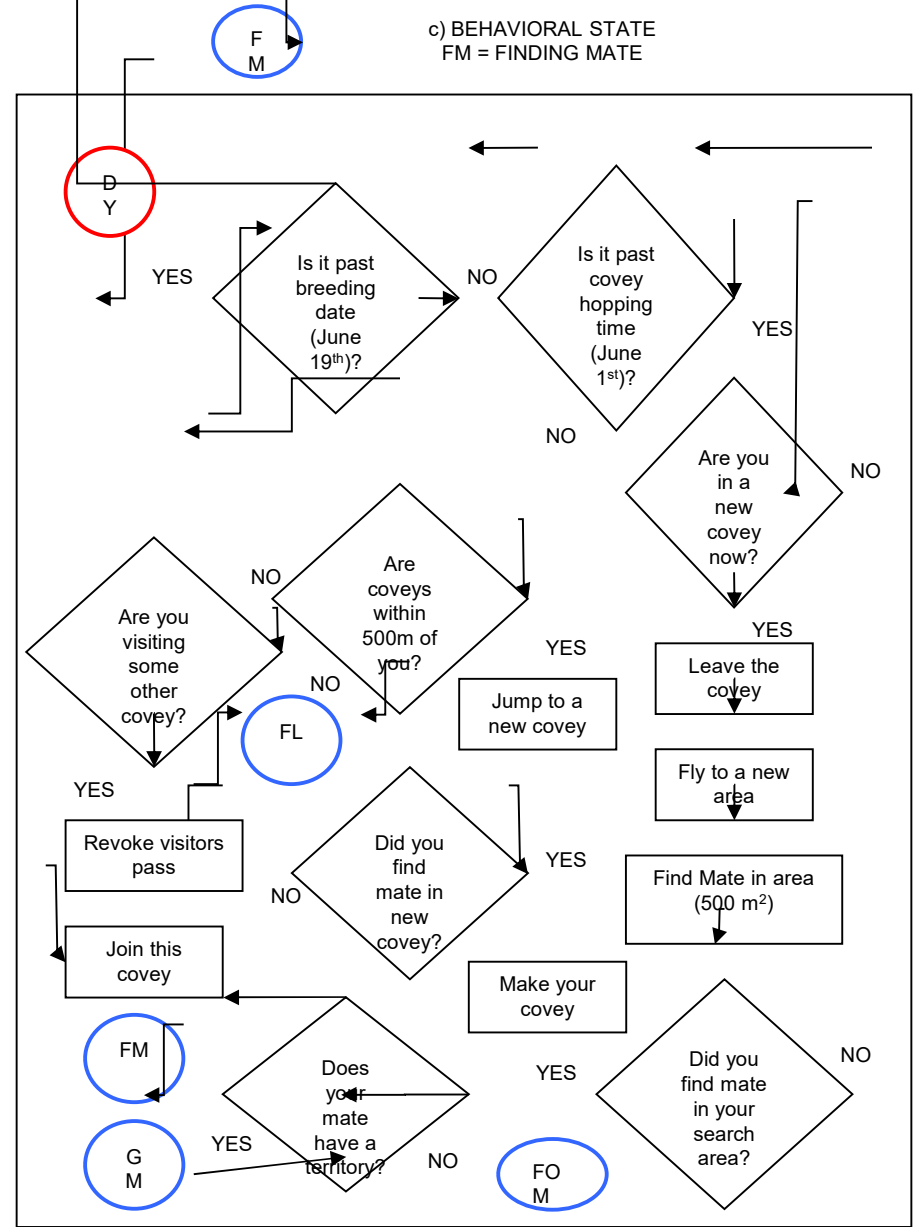
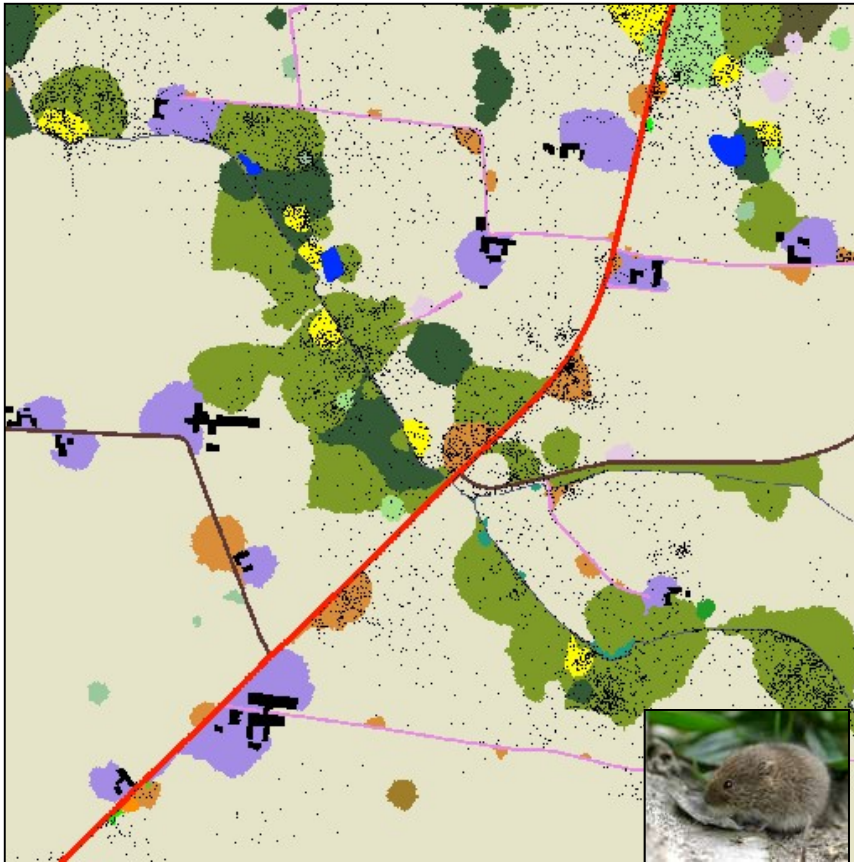
<https://www.comses.net/resources/education/>

Spatially explicit model of animal behaviour

Sibly, R.M., Akçakaya, H.R., Topping, C.J., O'Connor, R.J. (2005)
Population-level assessment of risks of pesticides to birds and
mammals in the UK. *Ecotoxicology*, 14, 863-876.

Population dynamics emerge as result of
local interactions, including pesticides.

Dynamic landscape with crop rotation and
weather-dependent plant growth



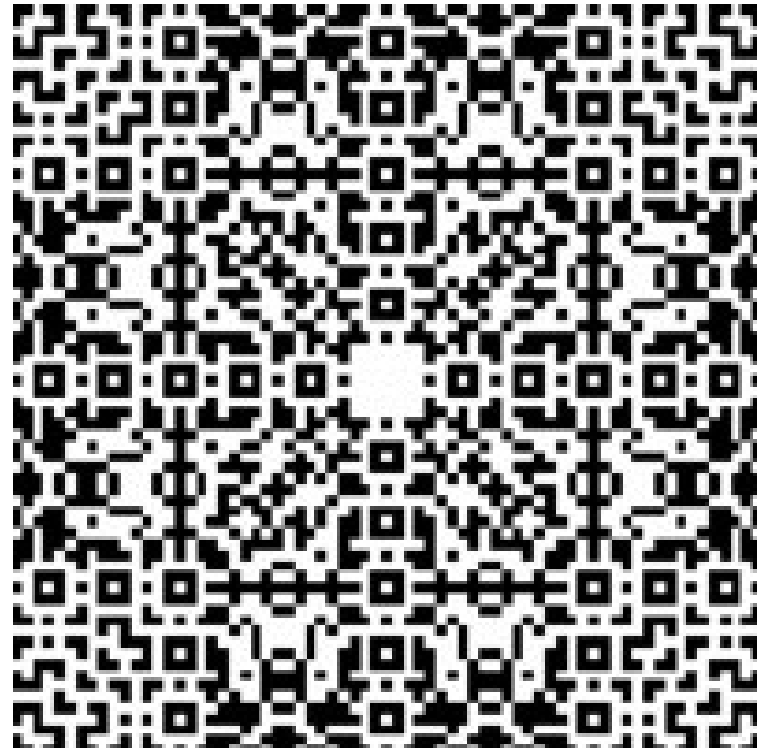
Adoption of Innovations: CA meets ABM

<http://ccl.northwestern.edu/netlogo/models/community/>

Parity Model

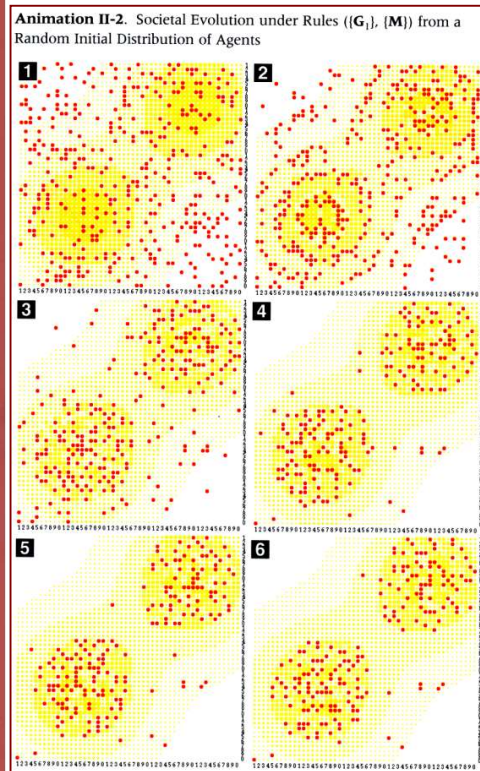
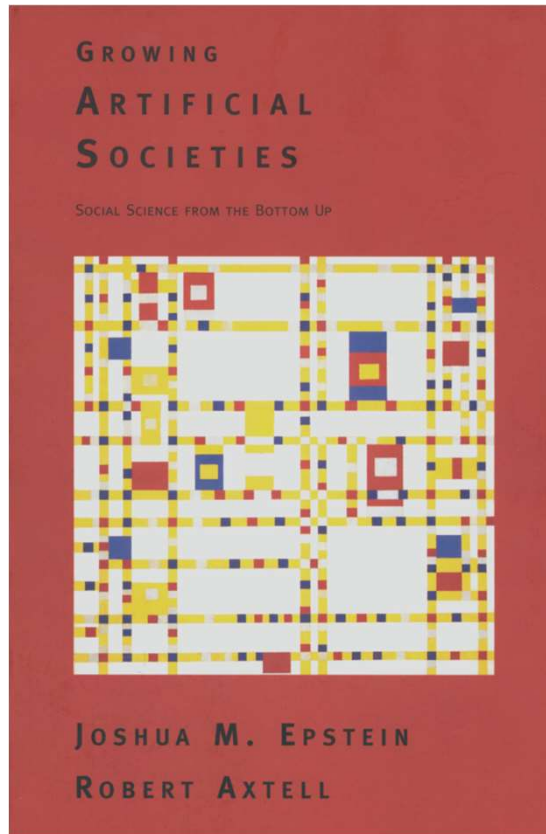
There is only one rule, which all the adopters follow.

- The rule is that each cell looks at its four neighbours (the ones immediately to its left, right, above and below)
 - if an odd number (that is, 1 or 3) have adopted, it adopts or turns 'on'.
 - if an even number have adopted (2 or 4), the agent does not adopt or is 'off'



Nigel Gilbert (2002)

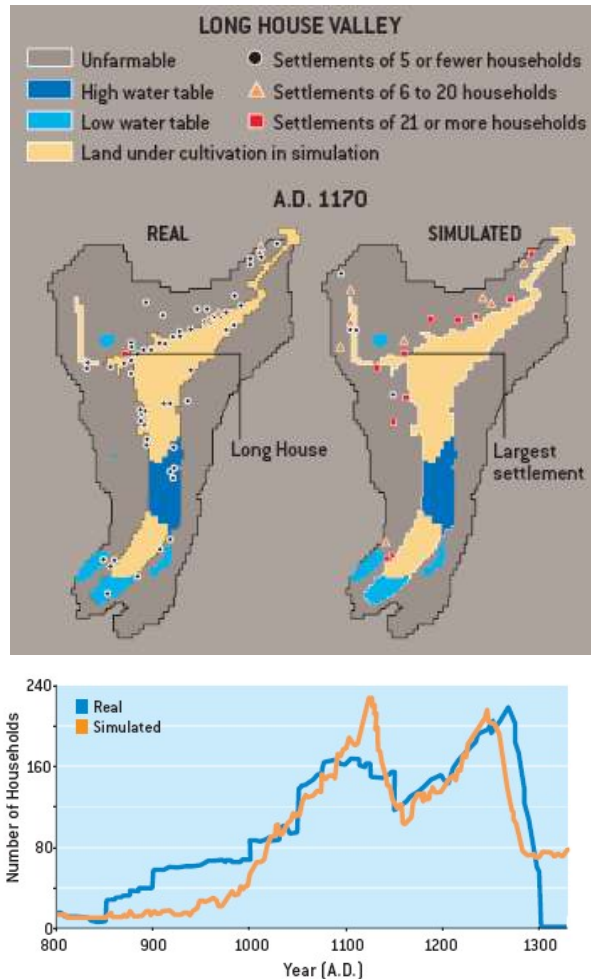
Sugarscape Was the First Comprehensive Computational Study of Artificial Societies



- ***Sugarscape Agents Have...***
 - ***Life, Death, Disease***
 - ***Trade: Sugar, Spice***
 - ***Wealth***
 - ***Sex, Reproduction***
 - ***Culture***
 - ***Conflict, War***
 - ***Externalities: Pollution***

The Puebloan Agent Model Traces Household Movements As They Seek The Best Plots for Growing Maize

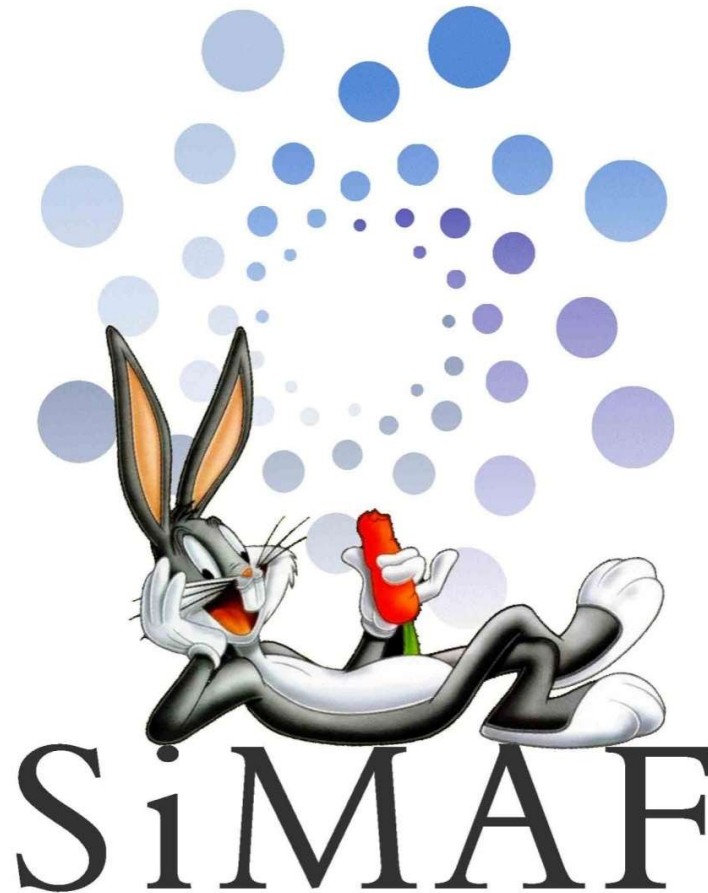
COPYRIGHT 2005 SCIENTIFIC AMERICAN, INC.



- Researchers constructed the model by entering environmental data—precipitation, water-table fluctuations etc. — on a digitized map of the valley.
- By A.D. 1170, the simulation shows the population clustering along the valley's northwestern margin, matching the actual pattern found by archaeologists.
- Although the simulated settlements are more aggregated than the real ones, the location of the largest settlement in the simulation is within 100 meters of the valley's biggest ruin, the Long House.

Kohler, Timothy A., George J. Gumerman and Robert G. Reynolds, "Simulating Ancient Societies," *Scientific American*, July 2005.

That's all folks!!



Questions?

Simulations in Materials Science, Astrophysics, and Physics

<https://sites.google.com/site/simafweb>

Web master: M.J. Erquiaga; design: E. Rim